

Compra desde la app

Dónde quedó el proyecto al 30/08/2026 y cómo seguir. Complementa el `COMPRA_EN_APP_SPEC.md`, que sigue siendo la fuente de verdad y tiene el detalle de cada decisión.

El 30 de agosto salió la primera compra completa desde la app, contra la tienda de demostración: carrito, cupón emitido de verdad, checkout con el descuento aplicado solo, y la pantalla de confirmación de AME. La clienta nunca vio la página de gracias de Tienda Nube.

Con eso quedó verificada la **detección del retorno**, que era el interrogante que el spec arrastraba desde el principio. El backend tiene **548 tests pasando**.

Lo que está hecho y probado

Todo se ejercitó contra la tienda de demostración, que es gratis de por vida y permite simular compras.

La compra completa, con una orden real en Tienda Nube

HECHO

Recorrido entero desde la app. Tienda Nube devuelve el cupón con `used: 1` y la orden queda en su panel.

Lo que confirmó: que las URLs de retorno funcionan, y que la inyección del cupón anda sola — no apareció el cartel para pegarlo a mano. **Falta saber cuál de las tres URLs matcheó**, para dejar solo esa: una adivinanza de más podría cerrar el WebView antes de que la clienta pague.

Producción lista para recibir instalaciones

HECHO

Credenciales de Tienda Nube en Railway, URL de redirección apuntada al callback, los tres webhooks de privacidad registrados, y el cron de limpieza de cupones corriendo cada hora. Más un centro de pruebas propio e inactivo, que aísla los ensayos del catálogo de AME.

Verificado desde afuera: Tienda Nube respondió identificando la app, y los webhooks rechazan lo que no viene firmado.

La medición de cuánto vende la app

HECHO

Endpoint `/api/analytics/dashboard/ventas-app/` y sección en la página de Analytics del CRM: facturado de la app contra el resto, ticket promedio comparado, descuento otorgado, qué productos, qué clientas, y cuántas compras se empezaron y no se terminaron.

La pregunta que contesta no es «cuánto vendió la app» sino el **ticket promedio con cupón contra sin cupón**: distingue las compras que el descuento trajo de las que iban a pasar igual y solo le costaron margen al centro.

El emparejador ya no depende solo del nombre

HECHO

Ahora mira, en este orden: **SKU exacto** (certeza, no parecido), después el nombre, y el **precio como testigo** — un match que convence por nombre pero cuyo precio difiere más del 50% baja a decisión humana. El precio nunca frena un match por SKU.

Probado contra la tienda demo: pasó de 4 a 5 emparejamientos. El quinto es el que el nombre no podía resolver — «Bruma Hidratante Y Calmante - 150 ML» contra «Bruma Descongestiva x150ml», unidos por SKU. Eso es lo que va a importar en el catálogo real de AME, donde los nombres casi nunca coinciden palabra por palabra.

Dos comandos que quedaron disponibles:

- Ensayar la vuelta de una venta, sin que nadie compre:

```
python manage.py simular_venta_app --cupon APP-XXXXXXX
```

- Y deshacerlo, que borra la venta, sus transacciones y libera el cupón:

```
python manage.py simular_venta_app --cupon APP-XXXXXXX --deshacer
```

Hace falta porque una compra en la tienda demo nunca vuelve por Conto, que mira la cuenta de la tienda real. El simulador genera ingresos reales en el módulo financiero: se niega a correr con DEBUG apagado, y hay que deshacerlo siempre.

Por dónde seguir

En este orden. Lo primero es lo más barato y lo que más riesgo despeja.

1. Confirmar que Conto manda el código del cupón

TRÁMITE

Es lo más importante que queda. Toda la atribución de ventas descansa en un campo que el equipo de Conto agregó **sin poder compilar**, y avisó. Si nunca llegó, la medición muestra ceros para siempre y nadie se entera.

- Correrlo en la consola de Railway, en el servicio del backend:
`python manage.py verificar_conto`
- Buscar en la salida `ventas.cupon`, `ventas.descuento_cupon`, `ventas.origen_venta` y `ventas.app_origen`.
- Si alguna dice «falta», avisarle a Conto antes de seguir.

Es de solo lectura y no necesita ninguna venta. En la base de desarrollo no corre: el token de Conto se deprecó a propósito, porque Conto vive solo en producción.

2. Desinstalar y reinstalar la app en la tienda demo

TRÁMITE

Es lo único del ensayo que quedó sin hacer, y lo que prueba de verdad el callback de OAuth y el webhook de borrado: desinstalar hace que Tienda Nube dispare `store/redact`, y reinstalar manda un código de instalación real al callback.

- La instalación se arranca desde el admin de Django de producción, en «Instalaciones de Tienda Nube iniciadas». Al guardar redirige a Tienda Nube. Hay quince minutos para completarla.
- Qué mirar: que la integración quede sin token y desactivada al desinstalar, que los cupones ya emitidos **sobrevivan**, y que al reinstalar la tienda se vincule sola.

Ojo: reinstalar emite un token nuevo y mata el que tiene el backend de desarrollo. Después hay que re-vincularlo a mano con `vincular_tienda_nube --token ... --store-id ...`, sacando el token nuevo del admin de producción.

3. Las cuatro preguntas al panel de partners

TRÁMITE

La que importa es una: **¿se puede instalar una app «en desarrollo» en la tienda real de AME?** Si la respuesta es no, hay que homologar la app y cambia el cronograma. Las otras tres: si una app «para tus clientes» pasa por homologación, si NubeSDK alcanza a una app que no inyecta nada, y el costo por transacción del plan real del centro.

Del costo por transacción ya hay un indicio fuerte: el panel de la tienda demo muestra **CPT 2%** para los medios de pago personalizados. Es el extremo alto del rango que se había estimado, y confirma la decisión de usar el checkout de Tienda Nube en vez de crear las órdenes por API.

4. Migrar conto-sync fuera de config-as-code

TRÁMITE

Railway deprecó los archivos de configuración: dejan de leerse el **01/12/2026**. Cuando llegue esa fecha, el sync de ventas se detiene, probablemente en silencio.

Es la misma configuración que ya se hizo para el cron de cupones —root directory, comando, política de reinicio «Never» y horario— cambiando el comando por `sincronizar_conto --que todo`.

Las dos preguntas del centro

No bloquean el trabajo técnico, pero una de las dos define si la app descuenta algo.

¿El descuento de la app se suma al de transferencia?

DECISIÓN

El centro ya da 10% por transferencia. Sobre un producto de \$20.900 con 15% de app, sumarlos es un 25% acumulado. Y son **dos perillas, no una**: el medio de pago del centro tiene su propia casilla de «combinar con otras promociones», aparte de la nuestra al crear el cupón.

Esto se puede medir en vez de preguntarlo: se pone el descuento por transferencia en 10% en la tienda demo, se compra con cupón y se mira el total. Si cobra 25% menos, se apilan. Así el centro decide sobre un número y no sobre una hipótesis.

Mientras no se responda, el segmento general queda en 0% y la app no descuenta nada — o sea que el incentivo para instalarla, que es el motivo del proyecto, no existe.

¿Las ofertas del CRM existen también en Tienda Nube?

DECISIÓN

Si el centro carga una oferta en el CRM y no en la tienda, la app muestra un precio con descuento que el checkout no va a respetar. Hoy no muerde porque ningún producto tiene oferta activa.

Si la respuesta es «no»: es cambiar una línea del serializer público para que la app muestre el precio de lista.

El último paso: la tienda real de AME

Nada acá se estrena. Todo se probó antes contra la de demostración.

Instalar la app y emparejar el catálogo

TRÁMITE

El catálogo no da trabajo: la tienda real *ya tiene* los productos, así que el emparejador los cruza contra el inventario de la plataforma.

- Correrlo primero en modo propuesta, que no escribe nada:
`python manage.py emparejar_variantes_tiendanube --centro`
- Revisar la lista. Los que dicen sku son certezas; los que muestran un porcentaje son parecidos de nombre.
- Aplicar cuando esté conforme, con `--aplicar`, y completar a mano desde el CRM los que quedaron sin emparejar.

Bloqueado por la pregunta de si una app «en desarrollo» se puede instalar en una tienda real.

Una compra real con un producto barato

TRÁMITE

Y después cancelarla, que devuelve el stock. Es lo único que confirma el último eslabón: que Conto manda el código del cupón en el comprobante y que la venta se atribuye a la app y a la clienta.

Avisarle al centro antes, para que no despachen una orden de prueba. Después de esto, la app vende.

Dos cosas anotadas que no son de este trabajo

Se encontraron construyendo y quedaron sin resolver a propósito.

cache_page cachea por URL, no por usuario

CÓDIGO

Todas las vistas de analytics lo usan. Se salvan a medias porque suelen recibir `sucursal_id` en la URL, pero una llamada sin ese parámetro es la misma URL para cualquier centro, y el primero en pedirla le deja sus números cacheados al siguiente.

Se descubrió porque los tests de la medición se pisaban entre sí. La vista nueva no lo usa; las anteriores sí.

Un job programado que apunta a código que no existe

CÓDIGO

`check-low-inventory` apunta a `apps.inventario.tasks`, que no existe. Ese job, programado todos los días a las 8, nunca corrió: la alerta de stock bajo solo se ve entrando al dashboard.

Celery no valida esos nombres al arrancar. Hay un test que importa todas las tasks del `beat_schedule` y falla el día que alguien la escriba, para que la excepción no quede olvidada.

Estado del repositorio. Todo el backend y el CRM están commiteados. El backend tiene **548 tests pasando**, y el typecheck del CRM y de la app están limpios. La base de desarrollo tiene la tienda demo vinculada con un token real y cinco productos emparejados, así que la compra desde la app se puede volver a ejercitar en cualquier momento.